

¿Qué es una Base de Datos?

Introducción: ¿Qué es “información”?

Para poder hablar de información primero necesitamos entender las piezas que la conforman: los **datos**. Y para entender **qué es un dato**, contamos con dos definiciones principales, que nos ayudarán a formar nuestra propia y práctica definición.

- Claude Shannon y Warren Weaver (1949): *“En particular, la información no debe confundirse con el significado. [...] Las señales o símbolos utilizados no necesitan tener ningún significado en absoluto; pueden ser simplemente elecciones entre un conjunto de alternativas posibles”*.
- Luciano Floridi (2011): *“De acuerdo con la Definición General de la Información (GDI), un dato es un recurso no interpretado que exhibe una falta de uniformidad. Formalmente: D es un dato si y solo si D consta de un número x de entidades no interpretadas, un número y de entidades no interpretadas, y una relación R entre x e y tal que x se distingue de y por ser diferente de y (es decir, $x \neq y$).”*

Luciano Floridi se acerca a la definición de dato desde un lado más fundamental, explicando que una falta de uniformidad genera un dato; mientras que Claude Shannon y Warren Weaver exponen que un dato es simplemente una señal o símbolo que, por sí mismo, probablemente no signifique nada. Un dato es la **unidad básica de información**.

¿Qué es una base de datos?

De igual manera, para entender qué es una base de datos partiremos de dos definiciones fundamentales:

- C. J. Date: *“Una base de datos es una colección de datos persistentes utilizados por los sistemas de aplicación de una empresa dada.”*
- Ramez Elmasri y Shamkant B. Navathe: *“Una base de datos es una colección de datos relacionados. Por datos entendemos hechos conocidos que pueden registrarse y que tienen un significado implícito. [...] Una base de datos tiene las siguientes propiedades implícitas:*
 - *Representa algún aspecto del mundo real*
 - *Es una colección lógicamente coherente de datos con algún significado inherente.*
 - *Se diseña, construye y puebla con datos para un propósito específico.”*

Estas dos definiciones, en realidad se complementan. Podemos concluir que una Base de Datos es entonces una colección de datos que representan algún aspecto del mundo real.

Arquitectura ANSI-SPARC

Es un modelo conceptual de referencia propuesto a mediados de la década de 1970 para organizar el diseño de los sistemas de bases de datos. Su propósito fue garantizar la **independencia de los datos** (tema que se explicará más a fondo en la siguiente sección).

Para lograr esta separación, el modelo propone dividir la arquitectura en **tres esquemas o niveles de abstracción**:

- **Esquema Externo (Nivel de Vista):** Describe la perspectiva particular de cada usuario o aplicación. Permite definir vistas parciales de la información, de modo que cada perfil interactúe únicamente con los datos pertinentes para sus tareas, manteniendo restringido el acceso al resto del sistema.
- **Esquema Conceptual (Nivel Lógico):** Representa la estructura global unificada de la base de datos. Define la totalidad de las entidades, sus atributos, las relaciones entre ellas y las restricciones de integridad. Sirve de puente intermedio y existe un único esquema conceptual para todo el sistema.
- **Esquema Interno (Nivel Físico):** Especifica cómo se organizan y almacenan físicamente los datos en los medios de almacenamiento secundario. Detalla las estructuras de archivos, los métodos de acceso, la indexación y la gestión del espacio en memoria.

Con esta división en tres capas, las modificaciones en la capa física no alteran la estructura lógica, y las actualizaciones en la estructura lógica no obligan a reprogramar las interfaces externas de los usuarios.

Independencia de datos

Es la capacidad de un sistema de bases de datos para cambiar la forma en que los datos están organizados o almacenados sin romper ni tener que reprogramar las aplicaciones que los utilizan.

Existen dos niveles:

- **Independencia Física de Datos:** Permite cambiar todo lo relacionado con el almacenamiento en el hardware (reorganizar archivos en disco, crear índices para acelerar búsquedas o mover los datos a otro servidor) sin modificar la estructura lógica de las tablas ni alterar las aplicaciones.
- **Independencia Lógica de Datos:** Permite modificar la estructura general de la base de datos (agregar nuevas tablas, añadir campos o modificar relaciones) sin afectar a los programas que solo leen o usan los campos antiguos.

Lenguajes de un sistema gestor

Son los conjuntos de comandos especializados que permiten interactuar con el sistema para construir su estructura, gestionar el contenido o controlar quién tiene acceso a la información. Formalmente se dividen en tres sublenguajes según el tipo de tarea que realizan:

1. DDL (*Data Definition Language* / Lenguaje de Definición de Datos)

Se encarga exclusivamente del **diseño y la estructura** de la base de datos (los esquemas). No sirve para modificar los registros individuales, sino para crear, alterar o borrar los contenedores donde reside la información.

- **Comandos representativos:** CREATE, ALTER, DROP, TRUNCATE.

2. DML (*Data Manipulation Language* / Lenguaje de Manipulación de Datos)

Se utiliza para **operar con el contenido** guardado dentro de las estructuras creadas por el DDL. Permite a los usuarios y programas hacer consultas y transformar la información diaria.

- **Comandos representativos:** SELECT, INSERT, UPDATE, DELETE.

3. DCL (*Data Control Language* / Lenguaje de Control de Datos)

Es la herramienta orientada a la **seguridad y permisos**. Determina las reglas de acceso para que cada rol o usuario del sistema solo pueda ver o ejecutar las acciones que le corresponden.

- **Comandos representativos:** GRANT, REVOKE.

Módulos componentes

Un Sistema Gestor de Bases de Datos (SGBD) se divide estructuralmente en dos grandes grupos de módulos: el **procesador de peticiones (motor de consultas)** y el **gestor de almacenamiento (motor de almacenamiento)**.

1. Procesador de Peticiones (Query Processor)

Es el conjunto de módulos que recibe las sentencias enviadas por los usuarios o aplicaciones (en DDL o DML), las analiza y las traduce en un plan de ejecución.

- **Intérprete DDL (DDL Interpreter):** Procesa los comandos de estructura (CREATE, ALTER, DROP) y actualiza los metadatos dentro del diccionario de datos o catálogo del sistema.
- **Compilador DML (DML Compiler):** Traduce las consultas e instrucciones de manipulación (SELECT, INSERT, UPDATE, DELETE) a instrucciones de bajo nivel reconocibles por la máquina.
- **Optimizador de Consultas (Query Optimizer):** Evalúa diferentes formas de ejecutar una consulta (planes de ejecución), calcula su costo computacional y selecciona la ruta más rápida y eficiente antes de ejecutarla.
- **Motor de Evaluación de Consultas (Query Evaluation Engine):** Ejecuta las instrucciones de bajo nivel generadas por el compilador y el optimizador para obtener o alterar los datos reales.

2. Gestor de Almacenamiento (Storage Manager)

Es la interfaz entre los datos de bajo nivel guardados en el disco duro o memoria física y las aplicaciones o consultas enviadas al sistema. Proporciona la abstracción necesaria para que el usuario no tenga que preocuparse por la ubicación física de los archivos.

- **Gestor de Archivos (File Manager):** Administra la asignación de espacio en disco y las estructuras de datos utilizadas para representar la información almacenada.
- **Gestor de Memoria Intermedia (Buffer Manager):** Controla el flujo de datos entre el almacenamiento secundario (disco) y la memoria principal (RAM), almacenando en caché la información de uso frecuente para acelerar la lectura y escritura.
- **Gestor de Transacciones (Transaction Manager):** Garantiza que las operaciones sobre la base de datos se ejecuten cumpliendo las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad), asegurando la integridad del sistema ante fallos.
- **Gestor de Autorización e Integridad (Authorization & Integrity Manager):** Verifica que el usuario tenga los permisos necesarios (DCL) para ejecutar la acción solicitada y valida que los datos cumplan con las restricciones del esquema (llaves primarias, foráneas, reglas).

Tipos de bases de datos

Los principales tipos de bases de datos se dividen según su modelo de datos y arquitectura, ofreciendo soluciones específicas para distintas necesidades:

Relacionales (RDBMS)

Organizan la información en tablas estructuradas por filas y columnas vinculadas mediante llaves primarias y foráneas. Garantizan integridad estricta mediante propiedades ACID.

- Caso de uso real: El sistema de transacciones bancarias de un banco (como BBVA o Santander), donde la consistencia de los saldos de cuenta al hacer transferencias es crítica.

Documentales (NoSQL)

Almacenan la información en documentos semiestructurados (generalmente formatos JSON o BSON), lo que permite esquemas flexibles que evolucionan dinámicamente.

- Caso de uso real: El catálogo de productos de una plataforma de comercio electrónico como Amazon, donde cada producto puede tener atributos completamente distintos (tallas de ropa frente a especificaciones técnicas de un CPU).

Clave-Valor (NoSQL)

Guardan datos mediante un esquema simple formado por una clave única vinculada a un valor. Destacan por su baja latencia y alta velocidad de lectura/escritura en memoria.

- Caso de uso real: La gestión de sesiones de usuario y carritos de compra activos en una plataforma como Netflix o Mercado Libre.

Familias de Columnas (Wide-Column)

Almacenan la información agrupada por columnas en lugar de filas, optimizadas para consultas analíticas sobre volúmenes masivos de datos (*Big Data*) distribuidos en clusters.

- Caso de uso real: La bandeja de entrada y la indexación masiva de correos electrónicos en Gmail.

De Grafos

Diseñadas para representar la información como nodos (entidades) y aristas (relaciones), priorizando la navegación veloz a través de estructuras altamente interconectadas.

- Caso de uso real: El motor de recomendaciones de conexiones y detección de fraude en redes sociales como LinkedIn o Facebook.

Especializadas

- Series de Tiempo (Time-Series): Optimizadas para registrar datos asociados a una marca temporal (*timestamp*) a alta velocidad.
 - Caso de uso real: El monitoreo continuo de telemetría en tiempo real de sensores IoT en plantas industriales o métricas de servidores.
- Espaciales: Diseñadas para almacenar, indexar y consultar información geográfica e imágenes vectoriales/rasterizadas (x, y, z , latitudes y longitudes).
 - Caso de uso real: El cálculo de rutas en aplicaciones de navegación y transporte geolocalizado como Google Maps o Uber.
- Vectoriales: Especializadas en almacenar vectores numéricos de alta dimensión (*embeddings*) para búsquedas por similitud semántica.
 - Caso de uso real: Los motores de búsqueda semántica y recuperación de información (RAG) en aplicaciones de inteligencia artificial generativa y Large Language Models (LLMs).

Clasificación de los sistemas gestores

Por el modelo de datos: Define cómo se representa la información. Los sistemas relacionales (RDBMS) usan tablas vinculadas con llaves para garantizar consistencia ACID. Los sistemas NoSQL emplean documentos (JSON), esquemas clave-valor, familias de columnas para analítica o grafos para evaluar conexiones. Existen también modelos jerárquicos, orientados a objetos y motores especializados para series de tiempo, datos espaciales o vectores.

Por el número de usuarios: Se determina por la capacidad del motor para gestionar la concurrencia. Los sistemas monousuarios atienden a un solo cliente a la vez en entornos locales o embebidos. Los sistemas multiusuario procesan miles de conexiones simultáneas mediante algoritmos de bloqueo y control de transacciones.

Por la arquitectura de distribución: Corresponde a la ubicación física del hardware y los archivos. En los sistemas centralizados todo reside en un solo servidor. En los distribuidos los datos se reparten entre múltiples nodos (homogéneos o heterogéneos) en red. En los federados varias bases de datos independientes cooperan manteniendo su autonomía.

Por el propósito: Depende del tipo de trabajo que optimiza el motor. Los SGBD de propósito general atienden cargas mixtas; los transaccionales (OLTP) se enfocan en lecturas y escrituras rápidas; los analíticos (OLAP / Data Warehouse) procesan consultas masivas sobre datos históricos; y los embebidos corren integrados en el código de una aplicación.

Por la licencia: Se refiere a los derechos de propiedad y acceso al código. El software de código abierto (Open Source) es libre de modificar y distribuir. El software propietario exige pagos de licencias y restringe el código. El modelo de código abierto comercial (Open-Core) ofrece un motor base gratuito y cobra por extensiones avanzadas.

Bases de datos en flujos de trabajo de inteligencia artificial

En los flujos de trabajo de inteligencia artificial, las bases de datos conectan los datos crudos con las decisiones del modelo. En la etapa inicial de entrenamiento, plataformas masivas como los Data Lakes o Data Warehouses guardan y organizan grandes volúmenes de datos no estructurados — como texto o imágenes— asegurando que la información esté limpia, ordenada y lista para alimentar a los modelos de machine learning.

Cuando el modelo ya está funcionando y responde a los usuarios, la base de datos actúa como una memoria externa mediante técnicas de recuperación de información conocidas como RAG. Si un usuario hace una pregunta, la base de datos busca los fragmentos más importantes en documentos corporativos y se los pasa al modelo en el momento, logrando que este dé respuestas precisas y actualizadas sin caer en alucinaciones ni requerir un reentrenamiento costoso.

Para que esta búsqueda sea realmente inteligente, entran en juego las bases de datos vectoriales. Estos motores transforman el texto o las imágenes en coordenadas matemáticas llamadas embeddings; así, en lugar de buscar palabras exactas como en una consulta tradicional de SQL, calculan distancias entre vectores usando métricas como la similitud de coseno para encontrar conceptos que significan lo mismo, aunque usen palabras totalmente distintas.

Referencias

Date, C. J. (2003). *An introduction to database systems* (8.^a ed.). Pearson.

Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of database systems* (7.^a ed.). Pearson.

Floridi, L. (2011). *The philosophy of information*. Oxford University Press.

Shannon, C. E., & Weaver, W. (1949). *The mathematical theory of communication*. The University of Illinois Press.

Tsichritzis, D. C., & Klug, A. (Eds.). (1978). ANSI/X3/SPARC DBMS framework report of the study group on database management systems. *Information Systems*, 3(3), 173-191.

Date, C. J. (2003). *An introduction to database systems* (8.^a ed.). Pearson. (*Véase el Capítulo 2: Database System Architecture*, pp. 31-45).

Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of database systems* (7.^a ed.). Pearson. (*Véase el Capítulo 2: Database System Concepts and Architecture*, pp. 31-34).

Oracle. (2019). *Database SQL Language Reference: Types of SQL statements* (19c). Oracle Help Center. <https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/Types-of-SQL-Statements.html>

Armbrust, M., Ghodsi, A., Xin, R., & Zaharia, M. (2021). Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. *Proceedings of CIDR 2021*. https://cidrdg.org/cidr2021/papers/cidr2021_paper17.pdf

Chen, P. P.-S. (1976). The entity-relationship model—toward a unified view of data. *ACM Transactions on Database Systems*, 1(1), 9–36. <https://doi.org/10.1145/320434.320440>

Codd, E. F. (1979). Extending the database relational model to capture more meaning. *ACM Transactions on Database Systems*, 4(4), 397–434. <https://doi.org/10.1145/320107.320109>

IBM. (2024). ¿Qué es una base de datos? IBM Think. <https://www.ibm.com/mx-es/think/topics/database>

Tsichritzis, D., & Klug, A. (Eds.). (1978). The ANSI/X3/SPARC DBMS framework report of the study group on database management systems. *Information Systems*, 3(3), 173–191. [https://doi.org/10.1016/0306-4379\(78\)90001-7](https://doi.org/10.1016/0306-4379(78)90001-7)

Universidad Internacional de La Rioja en México. (2024). Sistemas gestores de base de datos (SGBD): ¿Qué son y cuáles son los más usados? UNIR México. <https://mexico.unir.net/noticias/ingenieria/gestores-de-base-de-datos/>